

Full Stack Web Development



CHAPTER-4

JAVA STRUTS



Struts

The **struts 2 framework** is used to develop **MVC-based web application**.

The struts framework was initially created by **Craig McClanahan** and donated to Apache Foundation in May, 2000 and Struts 1.0 was released in June 2001.

The current stable release of Struts is Struts 2.3.16.1 in March 2, 2014.

This struts 2 tutorial covers all the topics of Struts 2 Framework with simplified examples for beginners and experienced persons.



Struts features

Struts 2 provides many features that were not in struts 1. The **important features** of struts 2 framework are as follows:

1. Configurable MVC components
2. POJO based actions
3. AJAX support
4. Integration support
5. Various Result Types
6. Various Tag support
7. Theme and Template support



Cont.

1) Configurable MVC components

In struts 2 framework, we provide all the components (view components and action) information in struts.xml file. If we need to change any information, we can simply change it in the xml file.

2) POJO based actions

In struts 2, action class is POJO (Plain Old Java Object) i.e. a simple java class. Here, you are not forced to implement any interface or inherit any class.

3) AJAX support

Struts 2 provides support to ajax technology. It is used to make asynchronous request i.e. it doesn't block the user. It sends only required field data to the server side not all. So it makes the performance fast.





Cont.

4) Integration Support

We can simply integrate the struts 2 application with hibernate, spring, tiles etc. frameworks.

5) Various Result Types

We can use JSP, freemarker, velocity etc. technologies as the result in struts 2.

6) Various Tag support

Struts 2 provides various types of tags such as UI tags, Data tags, control tags etc to ease the development of struts 2 application.

7) Theme and Template support

Struts 2 provides three types of theme support: xhtml, simple and css_xhtml. The xhtml is default theme of struts 2. Themes and templates can be used for common look and feel.



Steps to create Struts 2 Application Example

In this example, we are creating the struts 2 example without IDE. We can simply create the struts 2 application by following these simple steps:

1. Create the directory structure
2. Create input page (index.jsp)
3. Provide the entry of Controller in (web.xml) file
4. Create the action class (Product.java)
5. Map the request with the action in (struts.xml) file and define the view components
6. Create view components (welcome.jsp)
7. load the jar files
8. start server and deploy the project



Steps

1) Create the directory structure

The directory structure of struts 2 is same as servlet/JSP. Here, struts.xml file must be located in the classes folder.

2) Create input page (index.jsp)

This jsp page creates a form using struts UI tags. To use the struts UI tags, you need to specify uri /struts-tags. Here, we have used s:form to create a form, s:textfield to create a text field, s:submit to create a submit button.

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
2. `<s:form action="product">`
3. `<s:textfield name="id" label="Product Id"></s:textfield>`
4. `<s:textfield name="name" label="Product Name"></s:textfield>`
5. `<s:textfield name="price" label="Product Price"></s:textfield>`
6. `<s:submit value="save"></s:submit>`
7. `</s:form>`



Cont.

3) Provide the entry of Controller in (web.xml) file

In struts 2, **StrutsPrepareAndExecuteFilter** class works as the controller. As we know well, struts 2 uses filter for the controller. It is implicitly provided by the struts framework.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <web-app>
3.    <filter>
4.      <filter-name>struts2</filter-name>
5.      <filter-class>
6.        org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
7.      </filter-class>
8.    </filter>
9.    <filter-mapping>
10.     <filter-name>struts2</filter-name>
11.     <url-pattern>/*</url-pattern>
12.   </filter-mapping>
13. </web-app>
```



4) Create the action class (Product.java)

This is simple bean class. In struts 2, action is POJO (Plain Old Java Object). It has one extra method **execute** i.e. invoked by struts framework by default.

```
1. package com.javatpoint;
2.
3. public class Product {
4.     private int id;
5.     private String name;
6.     private float price;
7.     public int getId() {
8.         return id;
9.     }
10.    public void setId(int id) {
11.        this.id = id;
12.    }
13.    public String getName() {
14.        return name;
15.    }
```

```
1.    public void setName(String name) {
2.        this.name = name;
3.    }
4.    public float getPrice() {
5.        return price;
6.    }
7.    public void setPrice(float price) {
8.        this.price = price;
9.    }
10.
11.    public String execute(){
12.        return "success";
13.    }
14. }
```



5) Map the request in (struts.xml) file and define the view components

- It is the important file from where struts framework gets information about the action and decides which result to be invoked. Here, we have used many elements such as struts, package, action and result.
- **struts** element is the root elements of this file. It represents an application.
- **package** element is the sub element of struts. It represents a module of the application. It generally extends the **struts-default** package where many interceptors and result types are defined.
- **action** element is the sub element of package. It represents an action to be invoked for the incoming request. It has name, class and method attributes. If you don't specify name attribute by default execute() method will be invoked for the specified action class.
- **result** element is the sub element of action. It represents an view (result) that will be invoked. Struts framework checks the string returned by the action class, if it returns success, result page for the action is invoked whose name is success or has no name. It has **name** and **type** attributes. Both are optional. If you don't specify the result name, by default success is assumed as the result name. If you don't specify the type attribute, by default **dispatcher** is considered as the default result type.





struts.xml

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts
3. Configuration 2.1//EN" "http://struts.apache.org/dtds/struts-2.1.dtd">
4. <struts>
5. <package name="default" extends="struts-default">
6.
7. <action name="product" class="com.javatpoint.Product">
8. <result name="success">welcome.jsp</result>
9. </action>
10.
11. </package>
12. </struts>
```



6) Create view components (welcome.jsp)

It is the view component that displays information of the action. Here, we are using struts tags to get the information.

The `s:property` tag returns the value for the given name, stored in the action object.

welcome.jsp

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
- 2.
3. `Product Id:<s:property value="id"/><br/>`
4. `Product Name:<s:property value="name"/><br/>`
5. `Product Price:<s:property value="price"/><br/>`

7) Load the jar files

To run this application, you need to have the struts 2 jar files. Here, we are providing all the necessary jar files for struts 2. Download it and put these jar files in the lib folder of your project.

8) start server and deploy the project



Interceptor

Interceptor is an object that is invoked at the preprocessing and postprocessing of a request. In Struts 2, interceptor is used to perform operations such as validation, exception handling, internationalization, displaying intermediate result etc.

Advantage of interceptors

Pluggable If we need to remove any concern such as validation, exception handling, logging etc. from the application, we don't need to redeploy the application. We only need to remove the entry from the struts.xml file.

Struts 2 default interceptors

There are many interceptors provided by struts 2 framework. We have option to create our own interceptors. The struts 2 default interceptors are as follows:

- 1) alias** It converts similar parameters that have different names between requests.
- 2) autowiring**
- 3) chain** If it is used with chain result type, it makes the properties of previous action available in the current action.



Struts 2 Custom Interceptor Example

we can create the custom interceptor by implementing the Interceptor interface in a class and overriding its three life cycle method.

For creating the custom interceptor, **Interceptor** interface must be implemented. It has three methods:

1. **public void init()** It is invoked only once and used to initialize the interceptor.
2. **public String intercept(ActionInvocation ai)** It is invoked at each request, it is used to define the request processing logic. If it returns string, result page will be invoked, if it returns invoke() method of ActionInvocation interface, next interceptor or action will be invoked.
3. **public void destroy()** It is invoked only once and used to destroy the interceptor.

Example to create custom interceptor in struts 2

In this example, we are going to create custom interceptor that converts request processing data into uppercase letter.

You need to follow **2 steps** to create custom interceptor

1. **Create an interceptor (must implement Interceptor interface)**
2. **Define the entry of interceptor in the struts.xml file**



1) Create an interceptor (must implement Interceptor interface)

By this interceptor, we are converting the name property of action class into uppercase letter.

The **getStack()** method of ActionInvocation returns the reference of ValueStack.

We are getting the value set in the name property by **findString** method of ValueStack.

The **set** method of ValueStack sets the name property by the specified value. In such case, we are converting the value of name property into uppercase letter and storing it into the valuestack.

The **invoke** method of ActionInvocation returns the information of next resource.

MyInterceptor.java

```

1. package com;
2. import com.opensymphony.xwork2.ActionInvocation;
3. import com.opensymphony.xwork2.interceptor.Interceptor;
4. import com.opensymphony.xwork2.util.ValueStack;
5.
6. public class MyInterceptor implements Interceptor{
7.
8.     public void init() {}
9.     public String intercept(ActionInvocation ai) throws Exception {
10.         ValueStack stack=ai.getStack();
11.         String s=stack.findString("name");
12.         stack.set("name",s.toUpperCase());
13.         return ai.invoke();
14.     }
15.     public void destroy() {}
16. }
```



2) Define the entry of interceptor in the struts.xml file

To define the interceptor, we need to declare an interceptor first. The **interceptors** element of package is used to specify interceptors. The interceptor element of interceptors is used to define the custom interceptor. Here, we are defining the custom interceptor by upper.

The **interceptor-ref** subelement of action specifies the interceptor that will be applied for this action. Here, we are specifying the defaultstack interceptors and upper interceptor.

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts
3. Configuration 2.1//EN" "http://struts.apache.org/dtds/struts-2.1.dtd">
4. <struts>
5. <package name="abc" extends="struts-default">
6.
7. <interceptors>
8. <interceptor name="upper" class="com.MyInterceptor"></interceptor>
9. </interceptors>
```



Cont.

1. `<action name="login" class="com.Login">`
2. `<interceptor-ref name="defaultStack"></interceptor-ref>`
3. `<interceptor-ref name="upper"></interceptor-ref>`
4. `<result>welcome.jsp</result>`
5. `</action>`
- 6.
7. `</package>`
8. `</struts>`



Other Required Resources

The other required resources are

- index.jsp
- Login.java
- welcome.jsp

1) Create form to get input (index.jsp)

index.jsp

This jsp page creates a form using struts UI tags. It receives name from the user.

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
2. `<s:form action="login">`
3. `<s:textfield name="name" label="Name"></s:textfield>`
4. `<s:submit value="login"></s:submit>`
5. `</s:form>`



2) Create the action class

It is the simple action class containing name property with its setter and getter methods.

Login.java

```
1. package com;  
2. public class Login {  
3.     private String name;  
4.  
5.     public String getName() {  
6.         return name;  
7.     }  
8.     public void setName(String name) {  
9.         this.name = name;  
10.    }  
11.    public String execute(){  
12.        return "success";  
13.    }  
14. }
```



3) Create view component

This jsp page displays the name input by the user.
welcome.jsp

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
2. Welcome, `<s:property value="name"/>`

PU



params interceptor

The **params interceptor** also known as parameters interceptor is used to set all parameters on the valuestack.

It is found in the default stack by default. So you don't need to specify it explicitly.

Internal working of params interceptor

It gets all parameters by calling the **getParameters() method of ActionContext** and sets it on the valuestack by calling the **setValue() method of ValueStack**.





Parameters of params interceptor

There are 4 parameters defined for params interceptor

Parameter	Description
ordered	It is true by default but can be used to top-down the property setter behaviour.
paramNameMaxLength	specifies the maximum length for the parameter. The default length is 100 characters by default. The parameters length more than 100 will be ignored.
excludeParams	specifies the unallowed parameter names. You can specify multiple names separated with comma.
acceptParamNames	specifies the accepted parameter names.



Example of params interceptor

Let's see the simple example of params interceptor.

1. `<action name="login" class="com.javatpoint.Login">`
2. `<interceptor-ref name="params" />`
3. `<result name="success">login-success.jsp</result>`
4. `</action>`



Struts 2 defaultStack interceptors

The params interceptor is found in the default stack. You don't need to specify interceptors found in the default-stack. The defaultStack interceptors are as follows:

1. `<interceptor-stack name="defaultStack">`
2. `<interceptor-ref name="exception"/>`
3. `<interceptor-ref name="alias"/>`
4. `<interceptor-ref name="servletConfig"/>`
5. `<interceptor-ref name="prepare"/>`
6. `<interceptor-ref name="i18n"/>`
7. `<interceptor-ref name="chain"/>`
8. `<interceptor-ref name="debugging"/>`
9. `<interceptor-ref name="profiling"/>`
- 10.

cont.

1. `<interceptor-ref name="scopedModelDriven"/>`
2. `<interceptor-ref name="modelDriven"/>`
3. `<interceptor-ref name="fileUpload"/>`
4. `<interceptor-ref name="checkbox"/>`
5. `<interceptor-ref name="staticParams"/>`
6. `<interceptor-ref name="actionMappingParams"/>`
7. `<interceptor-ref name="params">`

Cont.

```
1. <param name="excludeParams">dojo\..*,^struts\..*</param>
2. </interceptor-ref>
3. <interceptor-ref name="conversionError"/>
4. <interceptor-ref name="validation">
5.   <param name="excludeMethods">input,back,cancel,browse</
param>
6. </interceptor-ref>
7. <interceptor-ref name="workflow">
8.   <param name="excludeMethods">input,back,cancel,browse</
param>
9. </interceptor-ref>
10. </interceptor-stack>
```

Struts 2 `execAndWait` interceptor

example

The `execAndWait` interceptor also known as execute and wait interceptor is used to display the intermediate result.

It is recommended to use for long running action.

It is not found in the default stack by default. So you need to specify it explicitly.

If you don't specify **"wait"** result, struts framework displays an intermediate result until your request is completed.

For the custom intermediate result, you need to define "wait" result in struts.xml file. In your page, you can display processing image etc. So, it is better to specify the custom result.

Parameters of execAndWait interceptor

There are 3 parameters defined for execAndWait interceptor.

Parameter	Description
delay	specifies the initial delay time. Bydefault, no initial delay is set.
delaySleepInterval	used only with delay. It specifies the time interval in milliseconds to check that background process is completed. It is set to 100 milliseconds bydefault.
threadPriority	specifies the priority of the thread. The default is Thread.NORM_PRIORITY.

Example of execAndWait interceptor without wait result

simple example of execAndWait interceptor without wait result. In such case, struts framework provides an intermediate result.

1. `<action name="login" class="com.javatpoint.Login">`
2. `<interceptor-ref name="params"/>`
3. `<interceptor-ref name="execAndWait"/>`
4. `<result name="success">login-success.jsp</result>`
5. `</action>`

Example of execAndWait interceptor with wait result

Let's see the simple example of execAndWait interceptor with wait result. In such case, your intermediate page is invoked.

1. `<action name="login" class="com.javatpoint.Login">`
2. `<interceptor-ref name="params"/>`
3. `<interceptor-ref name="execAndWait"/>`
4. `<result name="success">login-success.jsp</result>`
5. `<result name="wait">myintermediatepage.jsp</result>`
6. `</action>`



myintermediatepage.jsp

Let's write the code for intermediate result. The s:url tag will forward the request to specified url

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
2. `<html>`
3. `<head>`
4. `<title>wait</title>`
- 5.
6. `<meta http-equiv="refresh" content="0.5;url='<s:url includeParams="all" />'>`
7. `</head>`
8. `<body>`
- 9.
10. `<p>your request is processing...</p>`
11. ``
- 12.
13. `</body>`
14. `</html>`

Struts 2 prepare interceptor example

V

The **prepare interceptor** calls **prepare()** **method** on the action if it implements **Preparable interface**. It calls prepare() method before the execute() method.

To use the prepare interceptor, you need to implement Preparable interface in your action class and override its method prepare.

It is found in the default stack bydefault. So you don't need to specify it explicitly.

Parameters of prepare interceptor

There is only 1 parameter defined for prepare interceptor.

Parameter	Description
alwaysInvokePrepare	It is set to true bydefault.

Example of prepare interceptor

1. **<action name="login" class="com.javatpoint.LoginAction">**
2. **<interceptor-ref name="params"/>**
3. **<interceptor-ref name="prepare"/>**
4. **<result name="success">login-success.jsp</result>**
5. **</action>**

Action class

The action class must implement the Preparable interface and override its method prepare().

1. **package** com.javatpoint;
2. **import** com.opensymphony.xwork2.Preparable;
3. **public class** LoginAction **implements** Preparable{
4. **private** String name,password;

1. **public** String getPassword() {
2. **return** password;
3. }
4. **public void** setPassword(String password) {
5. **this**.password = password;
6. }
7. **public void** prepare() **throws** Exception {
8. System.out.println("preparation logic");
9. }
10. **public** String execute(){
11. System.out.println("actual logic");
12. **return** "success";
13. }
14. }
15. }

Struts 2 modelDriven interceptor example

The **modelDriven interceptor** makes other model object as the default object of valuelstack.

Bydefault, action object is the default object of valuelstack.

To use the modelDriven interceptor, you need to implement **ModelDriven interface** in your action class and override its method getModel().

It is found in the default stack bydefault. So you don't need to specify it explicitly.

Parameters of modelDriven interceptor

There is no parameter defined for modelDriven interceptor.

Example of modelDriven interceptor

1. `<action name="login" class="com.trainingtrains.LoginAction">`
2. `<interceptor-ref name="params"/>`
3. `<interceptor-ref name="modelDriven"/>`
4. `<result name="success">login-success.jsp</result>`
5. `</action>`

Full example of modelDriven interceptor

Let's see the full example of modelDriven interceptor.

File: *index.jsp*

```
1. <%@ taglib uri="/struts-tags" prefix="s" %>
2.
3. <s:form action="login">
4.   <s:textfield name="name" label="Name"></s:textfield>
5.   <s:password name="password" label="Password"></s:password>
6.   <s:submit value="login"></s:submit>
7. </s:form>
```

File: *struts.xml*

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN" "http://struts.apache.
   org/dtds/struts-2.1.dtd">
3. <struts>
4.
5.   <package name="abc" extends="struts-default" >
6.
7.     <action name="login" class="com.trainingtrains.Login">
       <result name="success">
```



File: Login.java

```
1. package com.trainingtrains;
2. import com.opensymphony.
   xwork2.ModelDriven;
3.
4. public class Login implements Model
   Driven<User>{
5.     private User user;
6.
7.     public User getUser() {
8.         return user;
9.     }
10.
11.    public void setUser(User user) {
12.        this.user = user;
13.    }
```

```
1.    public User getModel(){
2.        user=new User();
3.        return user;
4.    }
5.    public String execute(){
6.        if(user.getPassword().equals("admin")){
7.            return "success";
8.        }
9.        else{
10.            return "error";
11.        }
12.    }
13.
14. }
```



File: login-success.jsp

File: login-error.jsp

1. Sorry, username or password error!
2. `<jsp:include page="index.jsp"></jsp:include>`



Exception Handling - exception interceptor

In our web application, there might occur exception at any point.

To overcome this problem, struts 2 provides a mechanism of **global exception handling** where we can display a global result to the user.

Struts 2 automatically log the uncaught exceptions and redirects the user to the error handler page.

Understanding the internal working of exception interceptor

If there occurs exception, it is wrapped in **ExceptionHandler** and pushed in the valuestack so that we can easily access exception object from the result.

Note:- It is recommended to make this interceptor as the first interceptor, so that it can handle all the exception whether it is thrown by other interceptors.

Parameters of exception interceptor

There are 3 parameters defined for exception interceptor. All are optional.

Parameter	Description
logEnabled	specifies log should be enabled or not. You can pass true or false.
logLevel	specifies the log level. It may be trace, debug, info, warn, error, fatal. Default log level is debug.
logCategory.	specifies the log category eg. com.mycompany.app. The default is com.opensymphony.xwork2.interceptor.ExceptionMappingInterceptor..

Example of exception handling in struts 2

For exception handling, we specify the global-result and global-exception-mappings in the struts.xml file.

struts.xml

1. `<?xml version="1.0" encoding="UTF-8" ?>`
2. `<!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts`
3. `Configuration 2.1//EN" "http://struts.apache.org/dtds/struts-2.1.dtd">`
4. `<struts>`
5. `<package name="aa" extends="struts-default">`
- 6.
7. `<global-results>`
8. `<result name="myresult">globalresult.jsp</result>`
9. `</global-results>`
- 10.
- 11.

cont.

1. `<global-exception-mappings>`
2. `<exception-mapping result="myresult" exception="java.lang.
Exception"></exception-mapping>`
3. `</global-exception-mappings>`
- 4.
5. `<action name="login" class="com.Login">`
6. `<result>welcome.jsp</result>`
7. `<result name="error">error.jsp</result>`
8. `</action>`
- 9.
10. `</package>`
11. `</struts>`

Displaying exception

We can display the exception on the browser by printing the exception or exceptionStack.

The exception object prints the exception name whereas exceptionStack prints the exception details.

globalresult.jsp

1. `<p>Exception Name: <s:property value="exception" /> </p>`
2. `<p>Exception Details: <s:property value="exceptionStack" /></p>`

Full example of exception handling

The other required resources to complete this example are as follows:

1. **Input page (index.jsp)**
2. **Action class (Login.java)**
3. **View components (globalresult.jsp, welcome.jsp and error.jsp)**

1) Create index.jsp for input

This jsp page creates a form using struts UI tags. It receives name and password from the user.

index.jsp

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
- 2.
3. `<s:form action="login">`
4. `<s:textfield name="name" label="Name"></s:textfield>`
5. `<s:password name="password" label="Password"></s:password>`
6. `<s:submit value="login"></s:submit>`
7. `</s:form>`

2) Create the action class

This action class contains two fields name and password and one method execute. Here, we are throwing exception self if password matches struts.

This is dummy example, if you comment the mentioned line in the execute method, exception will not occur.



```
1. package com;
2. public class Login {
3.     private String name,password;
4.     //getters and setters
5.
6.     public String execute(){
7.         if(password.equals("struts")){
8.             int a=12/0;//If you comment this, exception will not occur
9.             return "success";
10.        }else
11.            return "error";
12.    }
13. }
```

3) Create view components

There are three view components globalresult.jsp that displays the exception message, welcome.jsp that displays the welcome message with the username and error.jsp that displays the error message.

globalresult.jsp

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
- 2.
3. Sorry an exception occurred!
4. `<p>Exception Name: <s:property value="exception" /> </p>`
5. `<p>Exception Details: <s:property value="exceptionStack" /></p>`
6. `<form>`
7. `<input type="button" value="back" onclick="history.back()">`
8. `</form>`

welcome.jsp

1. `<%@ taglib uri="/struts-tags" prefix="s" %>`
2. Welcome, `<s:property value="name"/>`

error.jsp

1. Sorry username or password error!
2. `<jsp:include page="index.jsp"></jsp:include>`

File Upload Example

The **fileUpload** interceptor automatically works for all the requests that includes files.

We can use this interceptor to control the working of file upload in struts2 such as defining allowed types, maximum file size etc.

Parameters of fileupload interceptor

There are 2 parameters defined for fileupload interceptor.

Parameter	Description
maximumSize	specifies maximum size of the file to be uploaded.
allowedTypes	specifies allowed types. It may be image/png, image/jpg etc.

It automatically adds 3 parameters in the request:

1. **File file** represents the file. We can apply methods on this object.
2. **String fileName** represents the filename of the file.
3. **String contentType** specifies the content type of the file.

Image upload example using struts 2



Let's see the **directory structure** of file upload application.

1) Create UserImage.jsp

This jsp page creates a form using struts UI tags. It receives name, password and email id from the user.

index.jspImage upload example using struts 2

Let's see the **directory structure** of file upload application.

1) Create UserImage.jsp

This jsp page creates a form using struts UI tags. It receives name, password and email id from the user.

cont.

index.jsp

1. `<%@ page contentType="text/html; charset=UTF-8"%>`
2. `<%@ taglib prefix="s" uri="/struts-tags"%>`
3. `<html>`
4. `<head>`
5. `<title>Upload User Image</title>`
6. `</head>`
7. `<body>`
8. `<h2>`
9. `Struts2 File Upload & Save Example without Database`
10. `</h2>`

2) Create SuccessUserImage.jsp

This jsp page creates a form using struts UI tags. It receives name, password and email id from the user.
SuccessUserImage.jsp

```
1. <%@ page contentType="text/html; charset=UTF-8"%><%@ taglib prefix="s"
2.   uri="/struts-tags"%>
3. <html>
4.   <head>
5.     <title>Success: Upload User Image</title>
6.   </head>
7.   <body>
8.     <h2>
9.       Struts2 File Upload Example
10.    </h2>
11.
```

cont.

1. User Image: `<s:property value="userImage" /><br/>`
2. Content Type: `<s:property value="userImageContentType" /><br/>`
3. File Name: `<s:property value="userImageFileName" /><br/>`
4. Uploaded Image: ``
`>"`
5. `width="100" height="100" />`
6. `</body>`
7. `</html>`

3) Create the action class

This action class inherits the ActionSupport class and overrides the execute method.

RegisterAction.java

```
1.  package com.trainingtrains;
2.  import java.io.File;
3.  import javax.servlet.http.HttpServletRequest;
4.  import org.apache.commons.io.FileUtils;
5.  import org.apache.struts2.interceptor.ServletRequestAware;
6.  import com.opensymphony.xwork2.ActionSupport;
7.
8.  public class FileUploadAction extends ActionSupport implements
9.      ServletRequestAware {
10.     private File userImage;
11.     private String userImageContentType;
12.     private String userImageFileName;
```

cont.

```
1.  private HttpServletRequest servletRequest;
2.  public String execute() {
3.      try {
4.          String filePath = servletRequest.getSession().getServletContext().getRealPath("/").
concat("userimages");
5.
6.          System.out.println("Image Location:" + filePath);//
see the server console for actual location
7.          File fileToCreate = new File(filePath, this.userImageFileName);
8.          FileUtils.copyFile(this.userImage, fileToCreate);//copying image in the new file
9.
10.         return SUCCESS;
11.     }
```

Cont.

```
1. public File getUserImage() {  
2.     return userImage;  
3. }  
4. public void setUserImage(File userImage) {  
5.     this.userImage = userImage;  
6. }  
7. public String getUserImageContentType() {  
8.     return userImageContentType;  
9. }  
10.  
11. public void setUserImageContentType(String userImageContentType) {  
12.     this.userImageContentType = userImageContentType;  
13. }  
14.
```

cont.

```
1.  public String getUserImageFileName() {  
2.      return userImageFileName;  
3.  }  
4.  public void setUserImageFileName(String userImageFileName) {  
5.      this.userImageFileName = userImageFileName;  
6.  }  
7.  public void setServletRequest(HttpServletRequest servletRequest) {  
8.      this.servletRequest = servletRequest;  
9.  }  
10. }  
11. }
```


4) Create struts.xml

This xml file defines an extra result by the name input, and an interceptor jsonValidatorWorkflowStack.

struts.xml

```
1. <!DOCTYPE struts PUBLIC
2.  "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
3.  "http://struts.apache.org/dtds/struts-2.0.dtd">
4.
5.
6.  <struts>
7.    <package name="fileUploadPackage" extends="struts-default">
8.      <action name="userImage" class="com.trainingtrains.FileUploadAction">
9.        <interceptor-ref name="fileUpload">
10.          <param name="maximumSize">2097152</param>
11.
12.          <param name="allowedTypes">
13.            image/png,image/gif,image/jpeg,image/pjpeg
14.          </param>
15.
```

cont.

1. `</interceptor-ref>`
2. `<interceptor-ref name="defaultStack"></interceptor-ref>`
3. `<result name="success">SuccessUserImage.jsp</result>`
4. `<result name="input">UserImage.jsp</result>`
5. `</action>`
6. `</package>`
7. `</struts>`

ValueStack

A valueStack is simply a stack that contains application specific objects such as action objects and other model object.

At the execution time, action is placed on the top of the stack.

We can put objects in the valuestack, query it and delete it.

ValueStack Interface

The struts 2 framework provides an interface to deal with valuestack. It provides many useful methods.

Methods of ValueStack interface

There are many methods in ValueStack interface. The commonly used methods are as follows:

public String findString(String expr) finds the string by evaluating the given expression.

public Object findValue(String expr) finds the value by evaluating the specified expression.

public Object findValue(String expr, Class c) finds the value by evaluating the specified expression.

public Object peek() It returns the object located on the top of the stack.

public Object pop() It returns the object located on the top of the stack and removes it.

cont.

public void push(Object o) It puts the object on the top of the stack.

public void set(String key, Object value) It sets the object on the stack with the given key. It can be get by calling the findValue(key) method.

public int size() It returns the number of objects from the stack.

ActionContext

The `ActionContext` is a container of objects in which action is executed. The values stored in the `ActionContext` are unique per thread (i.e. `ThreadLocal`). So we don't need to make our action thread safe.

We can get the reference of `ActionContext` by calling the `getContext()` method of `ActionContext` class. It is a static factory method. For example:

1. `ActionContext context = ActionContext.getContext();`

ActionInvocation

- [Struts 2 ActionInvocation Tutorial](#)
- [ActionInvocation Interface](#)

The **ActionInvocation** represents the execution state of an action. It holds the action and interceptors objects.

ActionInvocation Interface

The struts framework provides **ActionInvocation interface** to deal with ActionInvocation. It provides many methods, some of them can be used to get the instance of ValueStack, ActionProxy, ActionContext, Result etc.

Methods of ActionInvocation Interface

The commonly used methods of ActionInvocation interface are as follows:



No.	Method	Description
1)	<code>public ActionContext getInvocationContext()</code>	returns the ActionContext object associated with the ActionInvocation.
2)	<code>public ActionProxy getProxy()</code>	returns the ActionProxy instance holding this ActionInvocation.
3)	<code>public ValueStack getStack()</code>	returns the instance of ValueStack.
4)	<code>public Action getAction()</code>	returns the instance of Action associated with this ActionInvocation.
5)	<code>public void invoke()</code>	invokes the next resource in processing this ActionInvocation.
6)	<code>public Result getResult()</code>	returns the instance of Result.

Architecture

The **architecture and flow of struts 2 application**, is combined with many components such as Controller, ActionProxy, ActionMapper, Configuration Manager, ActionInvocation, Inerceptor, Action, Result etc.

Here, we are going to understand the struts flow by 2 ways:

1. struts 2 basic flow
2. struts 2 standard architecture and flow provided by apache struts

Struts 2 basic flow

Let's try to understand the basic flow of struts 2 application by this simple figure:

1. User sends a request for the action
2. Controller invokes the ActionInvocation
3. ActionInvocation invokes each interceptors and action
4. A result is generated
5. The result is sent back to the ActionInvocation
6. A HttpServletResponse is generated
7. Response is sent to the user

Struts 2 standard flow (Struts 2 architecture)

Let's try to understand the standard architecture of struts 2 application by this simple figure:

1. User sends a request for the action
2. Container maps the request in the web.xml file and gets the class name of controller.
3. Container invokes the controller (StrutsPrepareAndExecuteFilter or FilterDispatcher). Since struts2.1, it is StrutsPrepareAndExecuteFilter. Before 2.1 it was FilterDispatcher.
4. Controller gets the information for the action from the ActionMapper
5. Controller invokes the ActionProxy
6. ActionProxy gets the information of action and interceptor stack from the configuration manager which gets the information from the struts.xml file.
7. ActionProxy forwards the request to the ActionInvocation
8. ActionInvocation invokes each interceptors and action
9. A result is generated
10. The result is sent back to the ActionInvocation
11. A HttpServletResponse is generated
12. Response is sent to the user

Configuration File

The struts application contains two main configuration files **struts.xml** file and **struts.properties** file.

The struts.properties file is used to override the default values of **default.xml** file provided by struts framework. So it is not mandatory. Mostly, you will not use struts.properties file. We will learn about it later.

Here, we are going to learn all about struts.xml file. First of all let us see the simple example of struts.xml file

struts.xml

1. `<?xml version="1.0" encoding="UTF-8" ?>`
2. `<!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN" "http://struts.apache.org/dtds/struts-2.1.dtd">`
3. `<struts>`
4. `<package name="default" extends="struts-default">`

cont.

1. `<action name="product" class="com.trainingtrains.Product">`
2. `<result name="success">welcome.jsp</result>`
3. `</action>`
- 4.
5. `</package>`
6. `</struts>`

PU

1) package element

We can easily divide our struts application into sub modules. The package element specifies a module. You can have one or more packages in the struts.xml file.

Attributes of package element

- **name** name is must for defining any package.
- **namespace** It is an optional attribute of package. If namespace is not present, / is assumed as the default namespace. In such case, to invoke the action class, you need this URI:
 1. /actionName.action

If you specify any namespace, you need this URI:

 2. /namespaceName/actionName.action
- **extends** The package element mostly extends the **struts-default** package where interceptors and result types are defined. If you extend struts-default, all the actions of this package can use the interceptors and result-types defined in the **struts-default.xml** file.

2) action element

The action is the subelement of package and represents an action.

Attributes of action element

- **name** name is must for defining any action.
- **class** class is the optional attribute of action. If you omit the class attribute, **ActionSupport** will be considered as the default action. A simple action may be as:
 1. `<action name="product">`
- **method** It is an optional attribute. If you don't specify method attribute, **execute** method will be considered as the method of action class. So this code:

cont.

1. `<action name="product" class="com.trainingtrains.Product">`

will be same as:

2. `<action name="product" class="com.trainingtrains.
Product" method="execute">`

•

If you want to invoke a particular method of the action, you need to use method attribute.

3) result element

It is the sub element of action that specifies where to forward the request for this action.

Attributes of result element

- **name** is the optional attribute. If you omit the name attribute, success is assumed as the default result name.
- **type** is the optional attribute. If you omit the type attribute, dispatcher is assumed as the default result type.

Other elements

There are many other elements also such as **global-exception-mappings**, **global-results**, **include** etc.

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



[www.paruluniversity.ac.](http://www.paruluniversity.ac.in)

[in](http://www.paruluniversity.ac.in)

